PanG2G: Scalable Base-Pair-Resolution Graph-to-Graph Alignment of Pangenome Graphs

Ahmad Shakleya and Suhaib A. Fahmy
King Abdullah University of Science and Technology, Thuwal, Saudi Arabia.
ahmad.shakleya@kaust.edu.sa

***Abstract*—Pangenome variation graphs have replaced the single linear reference as the state of the art for representing human genetic diversity, yet a basic operation is still missing: directly comparing two such graphs to each other. On population pangenomes the delta graph shows the population-specific structural variation that a sequence-to-reference method cannot express without a linear projection. We present PanG2G, the first system for base-pair resolution graph-to-graph alignment of pangenome variation graphs. In this work we characterize the workload, describe the system and validate the reference-anchored output on a planted ground truth. The reference-anchored diff scales near-linearly to an 8.2×10^9 bp graph pair ($\sim2.6\times$ the human genome) on a single core in 72 min with 100% precision/recall, and the parallel snarl-pairing engine reaches $50.1\times$ DP-phase speedup on 64 cores.**

***Index Terms*—comparative genomics, pangenome, variation graph, graph alignment, structural variation**

I. Introduction

Genome analysis has long relied on a single linear reference, which introduces a systematic bias: alleles absent from the reference are hard to detect [1]. Pangenome variation graphs remove this bias by encoding many haplotypes as paths through a graph whose bubbles make polymorphism explicit [2]; the Human Pangenome Reference Consortium (HPRC) draft, built from 47 diploid samples, demonstrates this at scale [3]. The availability of multiple population-specific pangenomes raises a natural question: how do two such graphs differ, and which alleles are private to one? Answering it requires aligning one graph to another. Global sequence-to-sequence [4] and sequence-to-graph alignment [1] are well established, but no tool performs graph-to-graph alignment at base-pair resolution; in practice, comparing two pangenomes still means projecting both onto a common linear reference, reintroducing the bias pangenomes were built to remove [3], [5].

In general, graph-to-graph alignment contains subgraph isomorphism and is therefore NP-hard [6]. The key observation is that variation graphs are not arbitrary graphs: they decompose into a hierarchy of snarls, the generalized bubbles of Paten et al. [7], and this snarl tree has bounded tree-width in practice [8]. PanG2G exploits this to factor the alignment into independent per-snarl bipartite-matching subproblems, each solved optimally in $O(k^3)$ by the Jonker-Volgenant linear assignment solver (LAPJV): given the $k\times k$ matrix of pairwise dissimilarities between the k alleles (paths through the bubble) of two snarls, it finds the one-to-one matching of minimum total cost via shortest augmenting paths [9]. Snarl pairing, however, is a linear-time heuristic, so end-to-end optimality is not guaranteed. The workload is irregular: most snarls are tiny single-nucleotide polymorphism (SNP) bubbles, but a few structural-variant regions are enormous, so per-snarl work is heavy-tailed and a static work partition is load-imbalanced. PanG2G addresses this with three systems techniques, each motivated by direct profiling of the workload: a cache-oblivious memory layout for the snarl tree, a minimizer index for anchor detection, and a priority-dispatch scheduler keyed on estimated subtree work.

The contributions made in this paper include:

- **A problem definition and working system.** We formally define base-pair-resolution graph-to-graph alignment for variation graphs (§III-A) and present PanG2G, the first system to address it. The delta graph it outputs is the graph-level analogue of a VCF: every allele present in one population graph but absent from the other.
- **A workload characterization that drives the design:** per-snarl work is heavy-tailed (Gini > 0.55) and scale-invariant.
- **A parallel, cache-oblivious implementation:** a C++20 engine with a van Emde Boas snarl-tree layout, minimizer anchor index, exact LAPJV matching and an OpenMP priority-dispatch runtime.
- **Evaluation on synthetic and real pangenomes:** a reference-anchored diff that scales near-linearly past whole-genome size, snarl-pairing error measured up to 10^5 snarls with an anchor-guided pairing that removes it, and real African vs. East Asian (1000 Genomes) population graphs whose delta is enriched near gnomAD-SV and ClinVar pathogenic sites ($p < 10^{-3}$).

II. Related Work

Global alignment beyond strings. The problem we address (§III-A) is a global alignment: every allele of one graph is either matched to an allele of the other or reported as a gap, the graph analogue of Needleman-Wunsch [4]. Partial-order alignment (POA) [10] generalizes that dynamic program to directed acyclic graphs and can align two partial-order graphs to each other, but it fills one table cell per pair of nodes, which is infeasible at 10^7 nodes. Exactness is also hard in principle: optimally aligning two multiple alignments is NP-complete [11], as is aligning a sequence to a graph that may itself be edited [12]. Scalable methods must therefore exploit

graph structure, as PanG2G does with snarls. Read mappers, whether to a sequence [13] or to a graph [14], [15], keep one input a string; they enter this paper only through the linearize-and-map workaround we compare against in §IV.

General graph comparison. Graph edit distance and biological network alignment compare arbitrary graphs. Exact graph edit distance is NP-hard; its standard approximation reduces it to a bipartite assignment between local substructures solved by Hungarian-type algorithms [16], the same reduction PanG2G applies inside each snarl. Network aligners such as IsoRank [17] match protein-interaction networks by combined topological and sequence similarity. Neither exploits the bubble structure of variation graphs or reports base-pair alleles. PanG2G confines the assignment to paired snarls, where it is small and solved exactly.

Pangenome graph comparison. Pangenome graphs are built by vg [14], minigraph [18], Minigraph-Cactus [19] and PGGB [20]. Comparing two of them today goes either through a reference, where `vg deconstruct` projects each graph's bubbles onto a reference path as a VCF, as in the HPRC release [3], and the two VCFs are then compared, or through coarse summaries. ODGI [21] computes path-level similarity within a single graph; PanGraph [22] and SplitMEM [23] relate pangenomes at gene-block or suffix-structure granularity, not at the allele level. The Pangenome Graph Variation Format (PGVF) [24], a GFA fork, has been proposed to represent graph-to-graph alignments, but no tool computes them. PanG2G is, to our knowledge, the first system that does so at allele resolution.

III. Methodology

A. Problem Definition

Definition 1 (Variation graph)**.** A variation graph $G = (V, E, \sigma, \mathscr{P})$ is a directed acyclic graph whose nodes carry sequence labels $\sigma : V \to \Sigma^+$, $\Sigma = \{\mathrm{A}, \mathrm{C}, \mathrm{G}, \mathrm{T}\}$, and whose haplotypes are the source-to-sink paths in $\mathscr{P}$. A snarl $S = (s,t)$ is a minimal pair of boundary nodes that separates the subgraph between them from the rest of G [7]; snarls nest into a snarl tree $T(G)$. An allele of S is an $s \to t$ path, read as the concatenated labels of its interior nodes; $\mathscr{A}(S)$ is the set of alleles of S and $k = |\mathscr{A}(S)|$.

Definition 2 (Graph-to-graph alignment)**.** Given variation graphs G_A, G_B with labels σ_A, σ_B, an alignment $\mathscr{M} = (M_v, M_e, M_s)$ consists of partial one-to-one maps between their nodes, edges and snarls such that (i) M_s preserves ancestry between $T(G_A)$ and $T(G_B)$ and the order of snarls along the matched backbone M_v, and (ii) each snarl pair $(S, S') \in M_s$ carries a partial one-to-one allele matching $\mu_{S,S'} \subseteq \mathscr{A}(S) \times \mathscr{A}(S')$. The problem is to find $\mathscr{M}^* = \arg\max_{\mathscr{M}} W(G_A, G_B, \mathscr{M})$, where

$$W(G_A, G_B, \mathscr{M}) = \alpha \sum_{(u,v) \in M_v} \mathrm{sim}(\sigma_A(u), \sigma_B(v)) + \beta \sum_{(e,e') \in M_e} \mathbf{1}[\text{cons.}] + \gamma \sum_{(S,S') \in M_s} \mathrm{SimBubble}(S, S'), \quad (1)$$

$\mathrm{sim}(s_1, s_2) = 1 - \mathrm{edit}(s_1, s_2)/\max(|s_1|, |s_2|)$, $\mathbf{1}[\text{cons.}]$ indicates that a matched edge joins matched endpoints, and $\mathrm{SimBubble}(S, S')$ is the summed similarity of the allele pairs in $\mu_{S,S'}$ minus a gap penalty per unmatched allele. The output is the delta graph $\Delta_{A \to B}$ (resp. $\Delta_{B \to A}$), the alleles of G_A (resp. G_B) left unmatched by $\mathscr{M}^*$, including every allele of a snarl without partner, and the shared alignment induced by the matched elements.

Condition (i) separates this problem from general graph matching: without it the problem contains subgraph isomorphism and is NP-hard [6]; with it the optimum decomposes over the snarl tree (2) and every leaf is an assignment problem. PanG2G solves each allele matching μ exactly and the snarl map M_s heuristically (§III-B), so it returns a feasible alignment whose within-snarl part is optimal but which need not equal $\mathscr{M}^*$. In the reference-anchored mode (§IV-C), M_s is instead fixed by shared reference coordinates and alleles match only when identical.

B. The Algorithm

PanG2G has five stages (Fig. 1): (1) detect anchors, nodes shared by both graphs; (2) decompose each graph into its snarl tree; (3) pair snarls across the graphs; (4) run a per-snarl-pair dynamic program whose leaf operation is allele matching; (5) emit the delta graphs $\Delta_{A \to B}, \Delta_{B \to A}$ (Fig. 2), the shared alignment, and a global score.

The formulation supports the nested recurrence (2), but the implementation and evaluation are depth-1: nested snarls are detected but only top-level snarls are kept, each with its $s \to t$ paths as alleles, and the graphs we evaluate (vg construct and synthetic generators) contain only simple SNP/indel bubbles; this paper reports no results on nested Minigraph-Cactus graphs. The recurrence, the van Emde Boas layout and the DAG dependencies are therefore exercised only at depth 1, where they reduce to independent per-snarl-pair assignments, and the numbers in §IV should be read with this scope in mind. In (1), (α, β, γ) default to $(1, 0.5, 2)$: the structural term is up-weighted because allele identity is the primary delta signal, and we do not sweep them because the delta depends only on the within-snarl matching μ, whose ranking of allele pairs is invariant to γ's scale. Sibling snarls are separated by their boundary nodes, so their subproblems are independent and each depth-1 pair is one $O(k^3)$ bipartite matching.

With nested children the score decomposes recursively,

$$\mathrm{OPT}(S_A, S_B) = W_{\text{seq}} + W_{\text{topo}} + \max_{\mu} \sum_{(c,c') \in \mu} \mathrm{OPT}(c, c'), \quad (2)$$

with μ over child-snarl matchings and the leaf case reducing to Algorithm 1; the implementation processes children before parents in van Emde Boas post-order, a path that is inert at depth 1 (confirmed by the vEB ablation, Fig. 7).

1) Anchor detection: Anchors are node pairs with in- and out-degree ≤ 1 in both graphs whose sequences are near-identical (sim ≥ 0.9); the degree rule admits the invariant backbone chain between bubbles and also the allele nodes of simple bubbles, so shared alleles act as anchors too. A naive all-pairs scan is quadratic ($\sim 10^{14}$ comparisons at human scale, $|V| \sim 10^7$); PanG2G instead builds a minimizer index

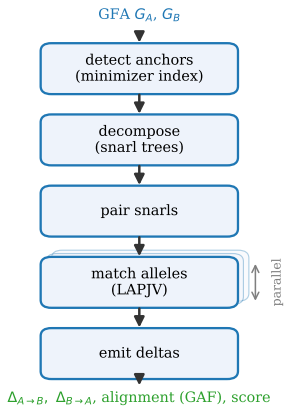


Fig. 1. The PanG2G pipeline. (1) Detect anchors via a minimizer index. (2) Decompose each graph into its snarl tree. (3) Pair snarls by boundary key (default) or anchor-interval dynamic programming. (4) Match alleles within each pair by LAPJV assignment. (5) Emit the delta graphs, the shared alignment (GAF-style) and a global score. Stage 4 runs in parallel (faded copies) under the priority-dispatch scheduler.

Algorithm 1 PanG2G alignment (**depth-1 form**, as implemented and evaluated). Each snarl pair is an independent leaf solved by one $O(k^3)$ assignment. $W_{\text{seq}} = \alpha \cdot$ mean similarity of the two snarls' source and sink nodes; $W_{\text{topo}} = \beta \cdot \mathbf{1}[\text{both are simple bubbles}]$. The shared alignment H (anchors, snarl pairs and their allele matchings) is written as a GAF-style alignment file, not materialized as a graph.

Require: graphs G_A, G_B; weights α, β, γ; gap g; floor τ
Ensure: deltas $\Delta_{A\to B}, \Delta_{B\to A}$, shared alignment H, score W

$\mathcal{N} \leftarrow \textsc{FindAnchors}(G_A, G_B)$ ▷ minimizer index
$T_A \leftarrow \textsc{Decompose}(G_A)$; $T_B \leftarrow \textsc{Decompose}(G_B)$ ▷ superbubbles
$P \leftarrow \textsc{PairSnarls}(T_A, T_B, \mathcal{N})$ ▷ boundary key or anchor-interval DP
$\mathcal{T} \leftarrow \textsc{BuildTaskDAG}(P)$ ▷ priority $= \sum k_i^3$
parfor $(S_A, S_B) \in \mathcal{T}$ **by priority dispatch do**
  $a \leftarrow \textsc{Alleles}(S_A)$; $b \leftarrow \textsc{Alleles}(S_B)$
  $C_{ij} \leftarrow 1 - \text{sim}(a_i, b_j)$; clamp $C_{ij} > 1-\tau$ to g
  $(s, \mu) \leftarrow \text{LAPJV}(C)$ ▷ $O(k^3)$ assignment
  $w[S_A, S_B] \leftarrow W_{\text{seq}} + W_{\text{topo}} + \gamma s$; record μ
end parfor
$W \leftarrow \sum w[S_A, S_B] + \alpha \sum_{(u,v)\in\mathcal{N}} \text{sim}(u, v)$
$U_A, U_B \leftarrow$ alleles unmatched by the μ's $\cup$ alleles of unpaired snarls
$\Delta_{A\to B}, \Delta_{B\to A} \leftarrow \textsc{BuildDeltas}(U_A, U_B)$
$H \leftarrow \mathcal{N} \cup P \cup \{\mu\}$ ▷ shared alignment
return $\Delta_{A\to B}, \Delta_{B\to A}$, H, W

[13] over one graph's eligible nodes: candidate pairs share a minimizer, are refined by full similarity and resolved greedily, and nodes shorter than the k-mer length (15 bp) produce no minimizer and are never anchored. On the vg construct graphs of §IV, half of all nodes satisfy the degree rule but over 80% of them are 1-bp SNP alleles, so anchors come from the ∼15% of eligible nodes of ≥15 bp, the reference chunks between variants (vg construct chops invariant sequence into nodes of at most 32 bp). Anchors contribute the α term of (1); the default pairing does not use them, and only the anchor-interval pairing below uses them to pin the two coordinate systems together. Anchoring is one-to-one, so differently cut nodes are not anchored; the reference-anchored mode (§IV-C), which keys variants by coordinate, is unaffected.

2) Snarl decomposition and pairing: PanG2G can take snarl boundaries from `vg snarls` [14] (optional backend), but by default uses a built-in superbubble scan after Onodera et al. [25], restricted to DAGs: from each branching node the scan walks the topological order, admitting a node once all its predecessors are visited, and closes the snarl at the first node that is the only frontier node left. Each scan costs the size of its bubble, so the decomposition is near-linear on bubble chains ($N^{1.00}$, Fig. 6). Both backends then enumerate every $s \to t$ path as an allele. Snarls are paired by their boundary key: two snarls correspond when the sequences of their source and sink nodes are identical, and repeated keys are consumed in topological order (FIFO). This rule is $O(|T_A| + |T_B|)$ and exact when keys are unique, but a snarl present in only one graph whose key repeats shifts every later snarl with that key by one position; on vg construct graphs 10–13% of bubbles share their key with another bubble, because 35–40% of boundary nodes are ≤2 bp. PanG2G therefore also offers an anchor-interval pairing (`-pairing interval`): anchors are chained in topological order and the snarls between two consecutive anchors are paired by an order-preserving dynamic program (a Needleman-Wunsch over the two snarl sequences) whose match score is the fraction of identical allele sequences plus the boundary similarity. An error can then not cross an anchor and the pairing is optimal within each interval; the cost is $\sum_i |A_i||B_i|$ over intervals, near-linear when anchors are dense. Both rules are compared in §IV (Q1). Unpaired snarls in either graph send all their alleles to that graph's delta.

3) Per-snarl bipartite matching: Within a paired snarl we want the maximum-similarity partial allele matching, with unmatched alleles paying a gap cost. PanG2G builds the $k \times k$ cost matrix $C_{ij} = 1 - \text{sim}(a_i, b_j)$, clamps pairs below a similarity floor ($\tau = 0.75$) to the gap cost so the solver prefers a gap over a bad match, pads rectangular instances with gap-cost dummies, and solves the assignment exactly with Jonker-Volgenant in $O(k^3)$ [9], so one pass optimizes matches and gaps jointly. For rare very large snarls ($k > 30$) a greedy 1/2-approximation is available as a fallback; a brute-force routine is retained only as a $k \leq 8$ oracle.

4) The output: The delta graph $\Delta_{A\to B}$ lists every allele present in A but unmatched in B (and symmetrically for $\Delta_{B\to A}$): the population-specific variants (Fig. 2), each with its exact inserted/deleted sequence. This is the graph-level analogue of a VCF, which no sequence-to-reference pipeline can produce without first collapsing both graphs onto a shared linear backbone.

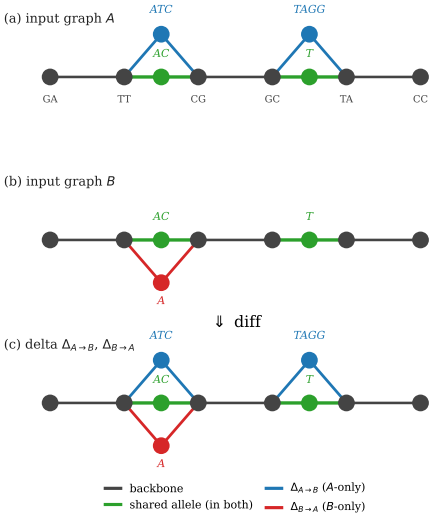


Fig. 2. The delta at base-pair resolution; every circle is a node storing a string of base pairs. Input graphs (a) A and (b) B share an allele (green) at each snarl and add their own alternative (blue for A, red for B). (c) the delta keeps the alleles in exactly one graph: $\Delta_{A\to B}$ (blue) and $\Delta_{B\to A}$ (red), each with its exact sequence.

C. Workload Profiling

Before building the parallel engine we profiled the reference prototype on synthetic graphs whose snarl sizes follow a Zipf law fit to HPRC statistics.

Finding 1: matching cost explodes with snarl size. Fig. 3b shows measured time per snarl pair as a function of allele count k. Brute-force enumeration ($O(k!)$) is fast for SNP bubbles but becomes infeasible beyond $k{\approx}8$ (57 ms per pair at $k{=}8$ in C++, 2 s in the Python prototype). The $O(k^3)$ LAPJV solver flattens this curve (1.2 ms at $k{=}50$) and is what makes complex structural-variant regions tractable, where k can reach 20-50 [9].

Finding 2: the workload is heavy-tailed and scale-invariant. Most snarls are tiny (67% are $k{=}2$ SNP bubbles) but a few are much larger, so per-snarl work ($\propto k^3$) is highly skewed. The Gini coefficient of work stays in the narrow band 0.55-0.58 across graph sizes spanning 20 to 1000 snarls (Fig. 3a; $k \leq 6$ in the prototype profiling): the imbalance does not wash out with scale. Real graphs have 10^6-10^7 snarls, so this is suggestive, not a deployment-scale demonstration. A static, equal-count partition of snarls across threads would be badly load-imbalanced, because a handful of large snarls dominate the work; this mandates a priority-dispatch scheduler.

Finding 3: memory access, not arithmetic, is the scaling wall. The snarl tree is traversed bottom-up and each snarl-pair dynamic program (DP) reads allele sequences from both graphs. At human scale the working set exceeds DRAM for a naive layout, which points to a cache-friendly layout for the snarl tree and to flat, contiguous sequence storage rather than pointer-chasing per-node strings.

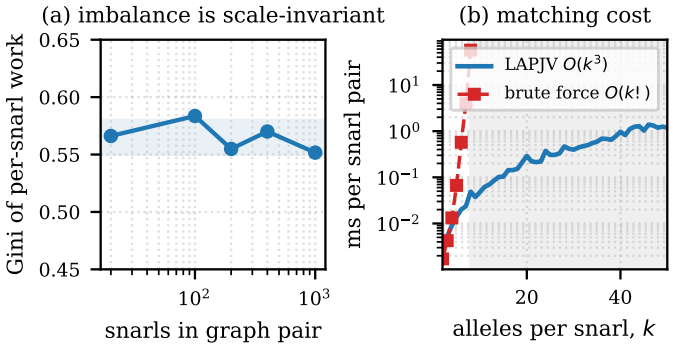


Fig. 3. (a) The Gini coefficient of per-snarl work stays above 0.55 across graph sizes (reference prototype, HPRC-calibrated synthetic graphs): the heavy tail is scale-invariant, so static partitioning is load-imbalanced at every scale. (b) Measured cost of the per-snarl bipartite matching versus allele count k (C++ engine, 12-bp alleles, mean of 20 pairs); brute force is infeasible beyond $k{=}8$.

D. System Design and Implementation

PanG2G is implemented in C++20 with a Python reference prototype as a correctness oracle (the prototype pairs snarls greedily by score over all pairs, which is quadratic; the engine uses the linear rules above). Graphs are read from GFA v1.1 (S/L/P and W lines, with on-the-fly reverse complementing). All node sequences live in one flat byte array indexed by (offset, length) and adjacency in CSR form, so the DP kernel streams sequence data without pointer chasing. Experiments run on a single-socket AMD EPYC 7763 (Milan): 64 cores, 4 NUMA domains, 512 GiB DDR4-3200, 256 MiB L3; Ubuntu 20.04, GCC 9.4 -O3, except where a laptop is named; the 1-64 core scaling run of Fig. effig:perfa used 64-core AMD EPYC 7702 nodes (IBEX).

Cache-oblivious snarl-tree layout. Each snarl's DP table and allele data sit contiguously in one arena block, which helps on all workloads and is always on. On a nested tree, breadth-first storage scatters sibling subtrees, so a bottom-up traversal thrashes the cache once the working set exceeds L3; PanG2G instead lays the tree out in van Emde Boas order [26], [27], storing any size-S subtree in $O(S/B)$ contiguous lines, so a bottom-up traversal is scan-optimal at $O(N/B)$ misses. All evaluated trees have depth 1, where vEB order $\equiv$ BFS order; the feature is implemented for the nested Minigraph-Cactus graphs that are the deployment target.

Centralized priority-dispatch runtime. Motivated by Finding 2, the snarl-pair DP is a shallow task DAG scheduled with Longest-Processing-Time-first (LPT) list scheduling [28] on OpenMP: a single mutex-guarded priority queue keyed on estimated subtree work ($\sum k_i^3$), from which idle workers pop the largest task. We chose one shared queue over per-thread work-stealing deques because randomized stealing loses the largest-first property LPT relies on; the price is lock contention, which may limit scaling at high thread counts (§IV, Q2). The parallel delta is byte-identical to the sequential aligner's and the score agrees to 10^{-3}, float-order aside.

Exact matching and anchor index. The solver is a self-

contained Jonker-Volgenant implementation [9]: the dispatcher routes $k \leq 30$ to LAPJV and larger snarls to the greedy fallback. The minimizer anchor index uses FNV-1a hashing over k-mers with a minimap2-style window [13], a build-time toggle against the naive scan.

IV. Results

A. Q1: Validation

We generate graph pairs with planted shared, A-only and B-only alleles and check that the deltas recover them, distinguishing three properties: (i) implementation determinism (the parallel engine reproduces the sequential result); (ii) optimization quality under the scoring model; (iii) biological truth, whether (α, β, γ) captures the correct correspondence, which is not formally established: the weights are hand-tuned and validated only indirectly, on the real graphs below.

Planted graphs. On planted pairs of 5 to 10^5 snarls (two shared and one private allele per side, 8-bp boundary nodes, three seeds each) both pairing rules recover the planted deltas with precision 1.000 and recall 0.997-1.000 on both delta sets; the residual is pairs of random 12-mers that fall within the $\tau = 0.75$ similarity floor and are matched, a property of the scoring model (iii), not of pairing. (The submitted version reported "$\geq 80\%$", the assertion threshold of the test suite rather than a measurement.) Within each paired snarl LAPJV recovers the planted alleles exactly and shared alleles never leak.

Pairing error at scale. The boundary-key rule can only err when boundary keys repeat and a snarl exists in one graph alone, which shifts the positional match of every later snarl with that key (§III-B). We measure this directly (Fig. 4): planted pairs with 1- or 2-bp boundary nodes, so that keys collide as on real vg construct graphs, 10% of snarl slots private to one graph, and a shared 24-bp linker node (an anchor) after every fourth snarl. The cascade is real and grows with size: with 1-bp boundaries the delta F_1 of the boundary rule falls from 1.00 at 5 snarls to 0.82 at 50, 0.63 at 10^3 and 0.55 at 10^5 (precision 0.38 at recall 0.998: every mispaired snarl dumps its alleles into both deltas), and with 2-bp boundaries from 0.99 at 50 to 0.57 at 10^5, whereas with unique 8-bp keys it stays at $F_1 \geq 0.9997$. The anchor-interval rule matches the unique-key rule row for row in every configuration up to 10^5 snarls, including 1-bp boundaries (precision 1.000; $F_1 \geq 0.998$ except one seed at 50 snarls, where a planted allele falls within the similarity floor and both rules miss it), and costs no measurable time (1.8-2.0 s single-threaded wall at 10^5 snarls for either rule, pairing included). Accuracy is thus validated at 10^5 snarls, one to two orders below deployment (10^6-10^7); the scaling runs reach 10^9 nodes but verify only parallel-vs-sequential consistency (property i), which both paths pass. Untested remain FNV-1a minimizer collisions beyond 10^8 k-mers and interleaved variant runs with more than 2^{16} alleles per snarl.

Real vg construct graphs. To test both modes on real graphs with a known answer we built two HLA-DRB1 pangenomes from six public haplotypes (three per graph; minimap2, paftools, vg construct against GRCh38): 1552 and 1279 atomic variants, 1275 shared, 277 A-only and 4 B-only. The reference-anchored diff recovers 273 of the 277 A-only variants (recall 0.99) at precision 0.78; every false positive lies in a cluster of adjacent indels and SNPs where the diff's edit-distance alignment places variants differently from the caller, the normalization limit of §IV-C, and two of the four B-only deletions sit in a tandem repeat where they are placed differently. The built-in superbubble finder and the `vg snarls` backend give byte-identical results. The first-common-successor finder of the submitted version closed a bubble one node early wherever vg construct links the alternate node of one SNP directly to the reference node of the next, and reached only recall 0.74 on the same graphs; the real-data runs below were therefore repeated. In the snarl-pairing mode, boundary keys pair 694 snarls and leave 470 and 330 unpaired (10-13% of keys repeat on these graphs), whereas the anchor-interval rule pairs 972, leaves 192 and 52 unpaired and raises the alignment score by 53%.

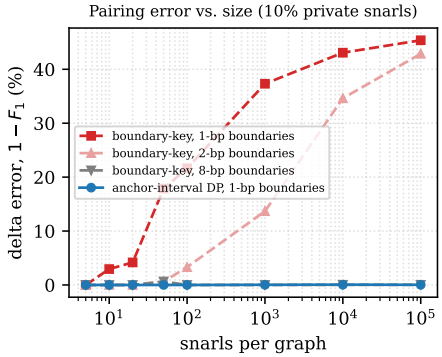


Fig. 4. Snarl-pairing error ($1 - F_1$ of the recovered delta, mean of both graphs and three seeds) versus graph size on planted graphs with 10% of snarls private to one graph. With repeated boundary keys (1-2 bp boundary nodes) the boundary-key rule's positional cascade grows with size; the anchor-interval dynamic program contains every error to one anchor interval.

Reference-anchored mode. Where snarls match by absolute coordinate, the mode satisfies property (ii) without approximation whenever both graphs embed the same reference with consistent representations: it achieves 100% precision/recall on every planted workload and self-diffs to empty.

B. Q2: Throughput and scaling

We measure parallel scaling, single-core scaling past whole-genome size, and throughput against a linearize-and-map baseline that extracts haplotypes and maps them to the reference with minimap2 [13]. Three ceilings bound the speedup. (1) End-to-end speedup saturates, at 4.49× on 64 cores, because the serial set-up (parsing, index build, decomposition, ∼4.6 s of the 22.1 s single-thread run) is not parallelized. (2) A critical-path bound: each snarl pair is one indivisible LAPJV call, so DP-phase speedup $\leq W_{\text{total}}/W_{\max}$ with work $\propto k^3$ and k up to 40, a ∼8,000× spread between the largest task and a k=2 bubble; the heavy tail that motivates dynamic scheduling (Finding 2) simultaneously caps it. (3) We evaluate up to 64 cores and make no claim beyond. All threads draw from one

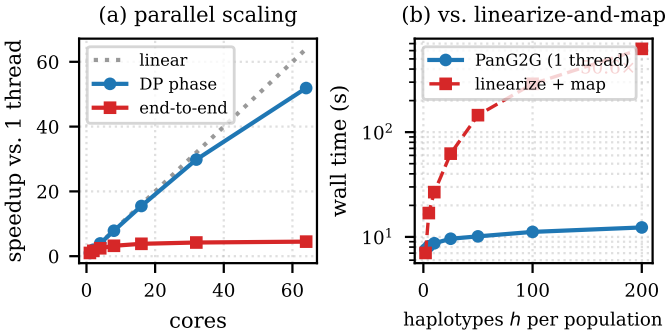


Fig. 5. (a) Parallel scaling to 64 cores (IBEX, EPYC 7702) on the 8-Mbp SV-like pair: DP-phase and end-to-end speedup, the latter Amdahl-capped by the sequential set-up; the harness flags the run invalid if parallel and sequential results ever diverge. (b) PanG2G's whole graph-to-graph diff vs. the extract+align cost of linearize-and-map on real chromosome-22 pangenomes as the per-population panel h grows: PanG2G (single-threaded) is sublinear in h (7.6→12.3 s over h=2→200), the baseline linear (7.0→624 s); they cross below h=5 and PanG2G is 51× faster at h=200.

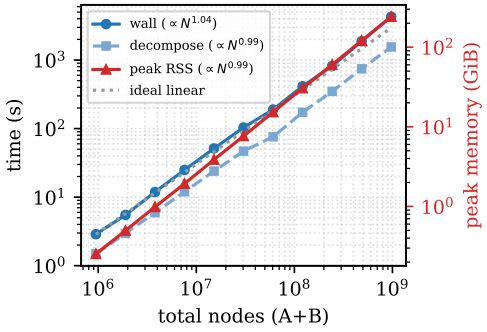


Fig. 6. Single-core scaling of the reference-anchored diff over 8×10^6-8.2×10^9 bp ($\sim10^6$-9.7×10^8 nodes): wall time $\propto N^{1.04}$ and snarl decomposition $\propto N^{1.00}$ (left axis), peak memory $\propto N^{0.99}$ (right axis); dotted: ideal linear. Exact at every size.

mutex-guarded queue, and half of the 25,608 tasks on this workload are k=2 bubbles, over which one lock acquisition per task is poorly amortized, so lock contention may limit larger thread counts. A variant that pops several of the largest ready tasks per lock acquisition gives no gain at 64 cores (53.4, 48.8 and 15.2× for chunks of 4, 8 and 32, against 52.0×), because a chunk hands one thread several of the largest tasks. The ablations (Q3) show that ordering by work (FIFO costs 1.20×) and dynamic dispatch (a static partition costs 1.92×) both matter. Up to 64 cores (81% DP-phase efficiency) DP-phase scaling is near-ideal: 2.0, 3.9, 7.9, 15.5, 29.8, 51.9× at 2-64 cores, byte-identical to the sequential result at every count (property i).

PanG2G and linearize-and-map compute different objects, a graph-to-graph delta versus per-haplotype alignments that still need calling and merging; no graph-to-graph tool exists to compare against, and timing only the baseline's alignment step and letting it multi-thread both favor it. PanG2G's cost is graph-size-dominated: even at h=2 the graph carries the full ∼51 Mb chr-22 backbone, so adding haplotypes only saturates shared variation and time grows sublinearly (7.6→12.3 s over h=2→200, single-threaded, superbubble finder) while the baseline is linear (7.0→624 s with 48 minimap2 threads). The curves cross below h=5; by h=200 PanG2G is 51× faster and widening (Fig. 5b), because it aligns the compressed graph directly instead of expanding it back to h sequences (the submitted version measured 12.7×; its decomposer dominated PanG2G's time here). A semi-linear baseline, mapping each haplotype of one population to the other population's graph with a sequence-to-graph mapper such as Giraffe [29], removes reference bias on the mapping side but still expands one graph into h sequences, so its cost should also grow linearly in h; we have not benchmarked it.

C. Reference-anchored diffing and single-node scaling

When both graphs embed a common reference, the usual case for vg construct or Minigraph-Cactus, PanG2G offers a second correspondence mode that replaces snarl pairing with matching by absolute reference coordinate. Each bubble is decomposed into atomic SNPs/indels keyed by reference position (*bubble_open* + *offset*); since the key depends only on the coordinate, a merged bubble in one graph yields the same atomic variants as separate bubbles in the other. This is invariant to node boundaries but not to variant representation: if two graphs place the same indel at different positions (left- vs. right-aligned in a homopolymer) the keys differ and a false delta results, as the HLA false positives show. PanG2G does not normalize variants, so same-tool graphs are consistent but mixed-provenance graphs need a normalization pass, and left-alignment is only its simplest part: builders such as Minigraph-Cactus and PGGB also cut a region into different snarls, nest variation differently, or merge and split adjacent SNPs. All comparisons in this paper are therefore between graphs from the same builder; cross-builder comparison requires decomposing each allele into a canonical set of primitive variants before diffing, which we leave to future work.

The mode scales near-linearly past whole-genome size on one core. We sweep synthetic pairs from 8×10^6 to 8.2×10^9 bp in which the two graphs cut the shared backbone differently (25 vs. 17 bp) and plant a known variant set with identical coordinate representations. PanG2G recovers the planted alleles exactly (100% precision/recall) and self-diffs to empty at every size; wall time and peak memory scale as $N^{1.04}$ and $N^{0.99}$, decomposition as $N^{1.00}$ (Fig. 6). The sweep reaches 8.2×10^9 bp (9.7×10^8 nodes, 4.1×10^7 variants, $\sim2.6\times$ the human genome) diffed exactly in 72 min at 242 GiB peak RSS and a constant 2.26×10^5 nodes/s, with 270 GiB still free. The sweep was timed with the windowed finder of the submitted version; the superbubble finder reports the same snarls and is faster (at 4 Gbp, the largest point within the re-measurement's 192 GB job limit: decomposition 51 instead of 745 s, wall 1475 instead of 1880 s, memory unchanged), so the wall times above are conservative; with decomposition no longer dominant the re-measured wall grows as $N^{1.12}$, the $N \log N$ of the coordinate-keyed variant maps.

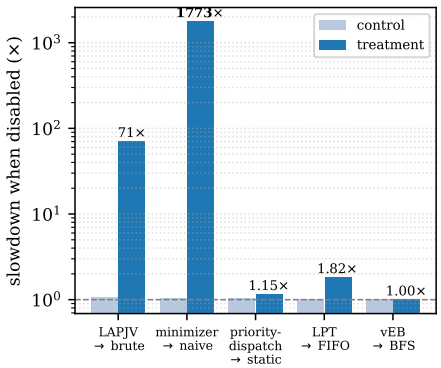


Fig. 7. Ablation of the five design choices, each disabled in isolation, at a matched control and treatment workload (log scale). LAPJV matching and the minimizer index are load-bearing (71×, 1773×); the scheduler ablations separate parallelism (static partition, 1.8×) from policy (FIFO, 1.15×); the vEB layout is a measured null on depth-1 snarls.

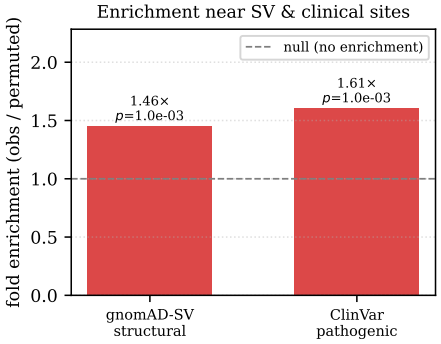


Fig. 8. PanG2G delta alleles on real AFR vs. EAS pangenomes (1000 Genomes phase-3, chromosome 22) are significantly enriched near gnomAD-SV structural variants and ClinVar pathogenic sites, both $p < 10^{-3}$ against a permutation null (fold and p annotated; dashed line: no enrichment).

D. Q3: Ablations

We disable each design choice in isolation (one compile-time flag each) and time the phase it affects at a control workload where the choice should be inert (balanced or L3-resident) and a treatment where it should bite (skewed, heavy-tailed or large-k; Fig. 7). Exact LAPJV matching and the minimizer index are the load-bearing choices: removing them inflates their phase by 71× and 1773× at treatment scale, since brute-force matching becomes infeasible as k grows (Finding 1) and the naive anchor scan dominates on large graphs. The scheduler ablations separate parallelism from policy: a static cyclic partition costs 1.82× on skewed work yet 1.00× on balanced work, confirming Finding 2, and downgrading LPT to FIFO costs a further 1.15× once per-snarl work exceeds scheduling overhead. The van Emde Boas layout reads 1.00× throughout: a genuine null on depth-1 snarls, whose subtrees fit a cache line either way. Every ablated binary returns a delta byte-identical to the baseline (property i).

E. Q4: Co-localization with structural and clinical variation

The decisive test of utility is whether the delta graph lands on biologically meaningful variation. We build two population pangenomes from the 1000 Genomes Project phase-3 panel (release 20130502): African (AFR, n=661) and East Asian (EAS, n=504), with vg construct on GRCh37 chromosome 22. We diff them with PanG2G's reference-coordinate mode and test whether the 608,718 delta alleles, 428k present only in the AFR graph and 181k only in EAS, fall closer (within 400 bp) than a uniform chromosome background to gnomAD-SV v2.1 [30] structural variants with AFR-EAS $F_{ST} \geq 0.2$ and to ClinVar [31] pathogenic sites: they do. The delta alleles are enriched 1.46× near gnomAD-SVs (946 observed vs. 650 expected; $p < 10^{-3}$, exceeding all 1000 permutations) and 1.61× near pathogenic sites ($p < 10^{-3}$; Fig. 8). The delta is therefore far from random; it concentrates in structural-variation and clinically relevant regions.

The enrichment is not specific to population-differentiated SVs: sweeping the gnomAD-SV F_{ST} threshold from 0.05 to 0.5 leaves the fold essentially flat (1.42-1.54×, every point $p < 10^{-3}$), so the chromosome-22 delta co-localizes with structural variation in general rather than preferentially with the most differentiated SVs. The cause is that the input graphs contain almost only SNVs/small indels: vg construct embeds only what the VCF provides, so the pangenomes are SV-depleted by construction; only 25 of the delta alleles insert or delete ≥ 50 bp, too few to test, and an SV-containing panel or a frequency-aware delta would be needed to isolate differentiation. What this experiment establishes is that the delta is biologically meaningful, not just a correct set difference: the graph-to-graph delta surfaces real structural and clinical variation directly, without the reference-projection step a sequence-to-reference pipeline would require.

V. Discussion

Every major decision in PanG2G traces to a measured property of the workload. Snarl-tree factorization makes an NP-hard problem tractable; LAPJV replaces brute-force matching because matching cost explodes with k (Finding 1); centralized priority dispatch replaces static partitioning because per-snarl work is heavy-tailed and scale-invariant (Finding 2), which the static-partition ablation confirms directly (Fig. 7); and the cache-oblivious layout plus flat sequence storage address the memory wall (Finding 3). The minimizer index and the superbubble scan remove the two originally quadratic stages, leaving an end-to-end pipeline that is near-linear in graph size (Fig. 6).

A. Limitations

The decomposer is DAG-only: although the vg-snarl backend can supply boundaries, allele enumeration and the DP assume acyclicity, so tandem repeats (cycles) are out of scope. The evaluation is depth-1 and same-builder only; nested Minigraph-Cactus graphs and cross-builder comparison, which needs variant normalization beyond left-alignment, remain to be tested, and the HLA experiment shows that even same-builder graphs diffed by coordinate inherit the caller's indel placement. Snarl pairing is a heuristic: the boundary-key rule drifts once keys repeat and snarls are private to one graph, the

anchor-interval rule removes that drift on our planted graphs but is optimal only within an interval, and a full tree-matching pairing remains the principled end point. Scaling beyond 64 cores is untested. PanG2G is single-machine; the $K > 2$ multi-graph problem, a distributed engine and GPU acceleration of the per-snarl matching, which is embarrassingly parallel across snarls, are natural extensions.

VI. Conclusion

We presented PanG2G, the first system that computes base-pair-resolution graph-to-graph alignments of pangenome variation graphs. The reference-anchored diff, the primary validated result, is exact under identical variant representation and scales near-linearly to an 8-Gbp pair on one core. The parallel snarl-pairing aligner carries the full systems specification (minimizer index, LAPJV matching, priority-dispatch scheduler, vEB layout), is validated on depth-1 workloads up to 10^5 snarls ($50.1\times$ DP-phase speedup at 64 cores, $51\times$ faster than linearize-and-map at h=200), and its anchor-interval pairing removes the positional drift of boundary-key pairing. A biological run confirms the delta is meaningful on real data. Graph-to-graph alignment is thus a viable pangenome-era primitive: the reference-anchored diff demonstrates it concretely, the parallel engine takes a first step toward the general case.

a) Availability.: Source code (MIT license), the Python prototype, the synthetic generators with the seeds used here, the experiment harnesses, the measured result tables and the figure script are at https://github.com/ahmadshakleya/G2G; the camera-ready release is archived at Zenodo (DOI: ZENODO-DOI-PLACEHOLDER). All real inputs are public: the 1000 Genomes phase-3 panel (release 20130502), GRCh37, gnomAD-SV v2.1 sites [30], the ClinVar GRCh37 VCF (weekly release of 28 September 2026) [31] and the PGGB HLA-DRB1 test haplotypes; the repository's scripts download them and rerun every experiment and figure reported here.

References

[1] Y. Cui *et al.*, "A survey of sequence-to-graph mapping algorithms in the pangenome era," *Genome Biology*, vol. 26, no. 1, p. 138, May 2025.

[2] E. Garrison *et al.*, "Building pangenome graphs," *Nature Methods*, vol. 21, no. 11, pp. 2008–2012, Nov 2024.

[3] W.-W. Liao *et al.*, "A draft human pangenome reference," *Nature*, vol. 617, no. 7960, pp. 312–324, May 2023.

[4] S. B. Needleman and C. D. Wunsch, "A general method applicable to the search for similarities in the amino acid sequence of two proteins," *Journal of Molecular Biology*, vol. 48, no. 3, pp. 443–453, 1970.

[5] J. M. Eizenga *et al.*, "Pangenome graphs," *Annual review of genomics and human genetics*, vol. 21, no. 1, pp. 139–162, 2020.

[6] A. Gupta and N. Nishimura, "The complexity of subgraph isomorphism for classes of partial k-trees," *Theoretical Computer Science*, vol. 164, no. 1, pp. 287–298, 1996.

[7] B. Paten *et al.*, "Superbubbles, ultrabubbles and cacti," in *Research in Computational Molecular Biology*, S. C. Sahinalp, Ed. Cham: Springer International Publishing, 2017, pp. 173–189.

[8] B. Brejová *et al.*, "Maximum-scoring path sets on pangenome graphs of constant treewidth," *Frontiers in Bioinformatics*, vol. Volume 4 - 2024, 2024. [Online]. Available: https://www.frontiersin.org/journals/bioinformatics/articles/10.3389/fbinf.2024.1391086

[9] R. Jonker and A. Volgenant, "A shortest augmenting path algorithm for dense and sparse linear assignment problems," *Computing*, vol. 38, no. 4, pp. 325–340, Dec 1987.

[10] C. Lee, C. Grasso, and M. F. Sharlow, "Multiple sequence alignment using partial order graphs," *Bioinformatics*, vol. 18, no. 3, pp. 452–464, 2002.

[11] J. Kececioglu and D. Starrett, "Aligning alignments exactly," in *Proceedings of the 8th Annual International Conference on Research in Computational Molecular Biology (RECOMB)*, 2004, pp. 85–96.

[12] C. Jain, H. Zhang, Y. Gao, and S. Aluru, "On the complexity of sequence-to-graph alignment," *Journal of Computational Biology*, vol. 27, no. 4, pp. 640–654, 2020.

[13] H. Li, "Minimap2: pairwise alignment for nucleotide sequences," *Bioinformatics*, vol. 34, no. 18, pp. 3094–3100, 09 2018.

[14] E. Garrison *et al.*, "Variation graph toolkit improves read mapping by representing genetic variation in the reference," *Nature Biotechnology*, vol. 36, no. 9, pp. 875–879, Sep 2018.

[15] A. Banerjee, D. Gibney, H. Xu, and S. Aluru, "A work-optimal parallel algorithm for aligning sequences to genome graphs," in *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, 2025, pp. 460–471.

[16] K. Riesen and H. Bunke, "Approximate graph edit distance computation by means of bipartite graph matching," *Image and Vision Computing*, vol. 27, no. 7, pp. 950–959, 2009.

[17] R. Singh, J. Xu, and B. Berger, "Global alignment of multiple protein interaction networks with application to functional orthology detection," *Proceedings of the National Academy of Sciences*, vol. 105, no. 35, pp. 12 763–12 768, 2008.

[18] H. Li, X. Feng, and C. Chu, "The design and construction of reference pangenome graphs with minigraph," *Genome Biology*, vol. 21, no. 1, p. 265, Oct 2020.

[19] G. Hickey *et al.*, "Pangenome graph construction from genome alignments with minigraph-cactus," *Nature Biotechnology*, vol. 42, no. 4, pp. 663–673, Apr 2024.

[20] E. Garrison and A. Guarracino, "Unbiased pangenome graphs," *Bioinformatics*, vol. 39, no. 1, p. btac743, 01 2023.

[21] A. Guarracino, S. Heumos, S. Nahnsen, P. Prins, and E. Garrison, "ODGI: understanding pangenome graphs," *Bioinformatics*, vol. 38, no. 13, pp. 3319–3326, 2022.

[22] N. Noll *et al.*, "PanGraph: scalable bacterial pan-genome graph construction," *Microb. Genom.*, vol. 9, no. 6, Jun. 2023.

[23] S. Marcus *et al.*, "Splitmem: a graphical algorithm for pan-genome analysis with suffix skips," *Bioinformatics*, vol. 30, no. 24, pp. 3476–3483, 12 2014.

[24] E. Garrison *et al.*, "Pangenome graph variation format (PGVF) specification," https://github.com/pangenome/pgvf-spec, 2023.

[25] T. Onodera, K. Sadakane, and T. Shibuya, "Detecting superbubbles in assembly graphs," in *Algorithms in Bioinformatics (WABI 2013)*, ser. Lecture Notes in Computer Science, vol. 8126. Springer, 2013, pp. 338–348.

[26] P. van Emde Boas, "Preserving order in a forest in less than logarithmic time and linear space," *Information Processing Letters*, vol. 6, no. 3, pp. 80–82, 1977.

[27] M. Frigo *et al.*, "Cache-oblivious algorithms," in *40th Annual Symposium on Foundations of Computer Science (Cat. No.99CB37039)*, 1999, pp. 285–297.

[28] R. L. Graham, "Bounds on multiprocessing timing anomalies," *SIAM J. Appl. Math.*, vol. 17, no. 2, p. 416–429, Mar. 1969.

[29] J. Sirén *et al.*, "Pangenomics enables genotyping of known structural variants in 5202 diverse genomes," *Science*, vol. 374, no. 6574, p. abg8871, Dec. 2021.

[30] R. L. Collins *et al.*, "A structural variation reference for medical and population genetics," *Nature*, vol. 581, no. 7809, pp. 444–451, May 2020.

[31] M. J. Landrum *et al.*, "ClinVar: improving access to variant interpretations and supporting evidence," *Nucleic Acids Res.*, vol. 46, no. D1, pp. D1062–D1067, Jan. 2018.